DATA STRUCTURES AND ALGORITHMS

(CS-218)

COMPLEX ENGINEERING ACTIVITY

BATCH: 2024

FALL SEMESTER 2025

GROUP MEMBERS:

NOOR UL AIN (CS-052)

AAISHA JAMIL (CS-053)

SYEDA MAHAM FATIMA (CS-054)

NAWAL ABID (CS-061)

Problem Statement:

This project requires a file compressor and decompressor capable of handling multiple types of files. This project involves developing a lossless file compression and decompression system capable of handling various file types, including both text and binary formats. The objective is to reduce storage requirements and optimize file transfer without compromising the integrity of the original data.

Solution Approach:

The solution involves designing an algorithm capable of compressing and decompressing files of various types while ensuring that the original data is fully preserved. The algorithm must handle variable-length encoding efficiently and retain all necessary information to accurately reconstruct the file during decompression.

Algorithm Selection:

For this project, Huffman Coding was chosen as the compression algorithm due to its efficiency, reliability, and suitability for lossless compression. Huffman Coding works by analyzing the frequency of each byte or symbol in a file and assigning shorter binary codes to frequently occurring bytes and longer codes to less frequent ones. This ensures optimal reduction of file size while preserving all original information.

Key Features:

- **Lossless Compression:** Guarantees that the original file can be perfectly reconstructed.
- **Frequency-Based Encoding:** More frequent symbols are encoded with fewer bits.
- **Adaptable to Multiple File Types:** Effective on text files, CSV, JSON, XML, and certain types of binary files.
- **Limitation:** Files with high randomness or pre-compressed formats (e.g., encrypted data or JPEGs) yield minimal compression.

Compression and Decompression Algorithm:

Compression Process:

The compression algorithm converts an input file into a compact encoded format while preserving all original data. The steps are:

- **Step 1: Frequency Analysis** – Read the input file and count the frequency of each byte.
- **Step 2: Huffman Tree Construction** – Create a binary tree where each leaf node represents a byte and its frequency. Combine the two least frequent nodes iteratively until a single root node is formed.
- **Step 3: Code Generation** – Assign variable-length binary codes to each byte based on the Huffman tree, ensuring more frequent bytes have shorter codes.
- **Step 4: Encoding** – Replace each byte in the input file with its corresponding Huffman code to generate the compressed data.
- **Step 5: Header Storage** – Save the Huffman tree or code table in the compressed file header for use during decompression.

Compression Code:

```python
import tkinter as tk
from tkinter import filedialog, ttk, messagebox
import os
import json
import heapq
import zlib
import sys
import subprocess

# --- Huffman Node ---
class Node:  5 usages
    def __init__(self,freq,char=None,left=None,right=None,codes={}):
        self.char=char
        self.freq=freq
        self.left=left
        self.right=right
        self.codes=codes
    def __lt__(self,other):
        return self.freq < other.freq

```

```python
# --- Compression ---
class file_compression:  1 usage
    def __init__(self):
        self.ch_freq = {}

    def file(self,chunk_size,original_file):
        '''Read the file in binary mode.Count occurrences of
        every byte.Save results to a JSON file.'''
        self.ch_freq = {}
        with open(original_file,"rb") as f:
            while True:
                chunk=f.read(chunk_size)
                if not chunk: break
                for i in chunk:
                    self.ch_freq[i]=self.ch_freq.get(i,0)+1
        with open("dp_1_frequency.json","w") as f:
            json.dump({str(k):v for k,v in self.ch_freq.items()}, f, indent=4)
```

```python

    def convert_heap(self):  1 usage
        '''this method turns the frequency dictionary into a list of Node objects,
        converts that list into a min-heap using heapq.heapify,
        and returns it for building the Huffman tree.'''
        min_heap=[Node(count,ch) for ch,count in self.ch_freq.items()]
        heapq.heapify(min_heap)
        return min_heap

    def build_huffman_tree(self):  2 usages
        '''creates a Huffman tree by repeatedly removing the two smallest-frequency
         nodes from the heap, merging them into a new parent node, and pushing that node back
          until only one node (the root) remains. It returns that root node.'''
        min_heap=self.convert_heap()
        if not min_heap: return None
        while len(min_heap)>1:
            node1=heapq.heappop(min_heap)
            node2=heapq.heappop(min_heap)
            heapq.heappush( *args: min_heap,Node(node1.freq+node2.freq, char: None,node1,node2))
        return min_heap[0]

```

```python

    def generate_codes(self, root=None):  1 usage
        ''' this method builds the Huffman tree (if not provided) and then performs a DFS traversal
        to assign binary codes to each character: left = "0", right = "1".
        Leaf nodes receive their final Huffman code, and all codes are stored in self.codes and returned.
        If the tree is empty, it returns an empty dictionary.'''
        if root is None:
            root = self.build_huffman_tree()
        codes={}
        if root is None:
            self.codes=codes
            return codes
        def dfs(node,prefix):
            if node is None: return
            if node.char is not None:
                codes[node.char] = prefix if prefix != "" else "0"
                return
            dfs(node.left,prefix+"0")
            dfs(node.right,prefix+"1")
        dfs(root, prefix: "")
        self.codes=codes
        return codes

```

```python

    def build_bitstring(self, input_path):  2 usages
        '''reads the input file byte-by-byte and replaces each byte with its
         Huffman code (from self.codes). It concatenates all those codes
          into one long bitstring and returns it'''
        if not getattr(self,'codes',None):
            raise ValueError("Codes not generated.")
        parts=[]
        with open(input_path,"rb") as f:
            data=f.read()
        for b in data:
            parts.append(self.codes[b])
        return "".join(parts)

    @staticmethod  1 usage
    def pad_bitstring(bitstring):
        '''pad_bitstring adds enough "0" bits to make the
        bitstring's length a multiple of 8. It returns the
         padded bitstring and the number of padding bits added.'''
        padding=(8-(len(bitstring)%8))%8
        if padding: bitstring+="0"*padding
        return bitstring,padding

```

```python
    @staticmethod  1 usage
    def bits_to_bytes(bitstring):
        '''bits_to_bytes takes the bitstring, processes it 8 bits
        at a time,converts each 8-bit chunk into an integer, appends
         it to a bytearray, and returns the final bytes object.'''
        out=bytearray()
        for i in range(0,len(bitstring),8):
            out.append(int(bitstring[i:i+8],2))
        return bytes(out)

    def write_huff_file(self,input_file,out_path="dp_1.huff",compress_header=True):  1 usage
        '''write_huff_file compresses a file using Huffman codes,
         optionally compresses the header, pads and converts the bitstring
         to bytes, and writes everything (header, padding, payload) to a .huff file.'''
        bitstring=self.build_bitstring(input_file)
        bitstring_padded,padding=self.pad_bitstring(bitstring)
        payload_bytes=self.bits_to_bytes(bitstring_padded)
        header_dict={str(k):v for k,v in self.codes.items()}
        header_bytes=json.dumps(header_dict).encode("utf-8")
```

```python
        header_bytes=json.dumps(header_dict).encode("utf-8")
        flag=0
        if compress_header:
            try:
                compressed=zlib.compress(header_bytes)
                if len(compressed)<len(header_bytes):
                    header_bytes=compressed
                    flag=1
            except: flag=0
        header_len=len(header_bytes)
        with open(out_path,"wb") as f:
            f.write(header_len.to_bytes( length: 4, byteorder: "big"))
            f.write(bytes([flag]))
            f.write(header_bytes)
            f.write(bytes([padding]))
            f.write(payload_bytes)
        return out_path
```

Decompression Process:

The decompression algorithm reconstructs the original file from the compressed data. The steps are:

- **Step 1: Read Header** – Retrieve the Huffman tree or code table from the compressed file header.
- **Step 2: Bit Stream Traversal** – Read the compressed bit stream sequentially.
- **Step 3: Huffman Tree Decoding** – Traverse the Huffman tree according to the bits until a leaf node is reached, identifying the original byte.
- **Step 4: Reconstruct File** – Repeat the traversal process until the entire bit stream is decoded, writing the original bytes to the output file.

Decompression code:

```python
# --- Decompression ---
class file_decompression:  1 usage

    def Read_header(self,huffman_file):  1 usage
        '''Read_header extracts the Huffman codes, compressed data,
        and padding length from a .huff file, decompressing the header if needed.'''
        with open(huffman_file,"rb") as f:
            data=f.read()
        header_len=int.from_bytes(data[:4], byteorder: "big")
        flag=data[4]
        header_start=5
        header_end=header_start+header_len
        header_bytes=data[header_start:header_end]
        if flag==1:
            header_json=zlib.decompress(header_bytes).decode("utf-8")
        else:
            header_json=header_bytes.decode("utf-8")
        codes=json.loads(header_json)
        padding_len=data[header_end]
        compressed_data=data[header_end+1:]
        return codes,compressed_data,padding_len

```

```python
    def Re_build_huffman(self,codes):  1 usage
        '''Re_build_huffman reconstructs the Huffman tree from
         a codes dictionary by creating nodes along the bit paths
          and assigning the byte value to each leaf.'''
        root=Node(0)
        for char_code,huff_code in codes.items():
            current=root
            byte_val=int(char_code)
            for bit in huff_code:
                if bit=="0":
                    if current.left is None:
                        current.left=Node(0)
                    current=current.left
                else:
                    if current.right is None:
                        current.right=Node(0)
                    current=current.right
            current.char=byte_val
        return root

    def bytes_to_bitstring(self,compressed_data,padding_len):  1 usage
        '''bytes_to_bitstring converts compressed bytes back
        into a bitstring and removes the padding bits added during compression.'''
```

```python
        parts=[format(byte,'08b') for byte in compressed_data]
        bitstring="".join(parts)
        if padding_len>0:
            bitstring=bitstring[:-padding_len]
        return bitstring

    def decompress(self,root,bitstring):  1 usage
        '''decompress reads the bitstring through the
        Huffman tree, outputs bytes at leaf nodes, and returns the decoded data.'''
        decoded=[]
        current_node=root
        for bit in bitstring:
            current_node=current_node.left if bit=="0" else current_node.right
            if current_node.char is not None:
                decoded.append(current_node.char)
                current_node=root
        return decoded

    def write_decoded_file(self,decoded_bytes,output_file):  1 usage
        '''write_decoded_file writes the list of
        decoded byte values to a file in binary mode.'''
        with open(output_file,"wb") as f:
            f.write(bytes(decoded_bytes))
```

Time and Space Complexity:

Time Complexity:

The total time complexity of the Huffman Compression algorithm comes from four major phases:

Let:

- **n** = size of the input file in bytes
- **k** = number of unique symbols in the file (for bytes, k≤256k \le 256k≤256)

Phase	Description	Complexity
Counting Frequencies	Scan the input file byte by byte to count symbol occurrences.	O(n)
Building the Min-Heap	Create a heap with k unique symbols.	O(k)
Constructing the Huffman Tree	Repeatedly extract two minimum nodes and merge them.	O(k log k)
Generating Codes	Traverse Huffman tree to assign codes.	O(k)
Encoding the File	Replace each byte in the file with its Huffman code.	O(n)
Writing Compressed File	Convert bitstring to bytes and write output.	O(n)

Overall Time Complexity:

$$T(n,k)=O(n)+O(k)+O(k\log k)+O(k)+O(n)+O(n)$$

The number of unique characters k is small and bounded (≤ 256 for standard ASCII text). Compared to the input size n, the contribution of k is negligible, so the effective time complexity is dominated by n.

The effective time complexity is:

$$\boxed{\mathbf{T(n)=O(n)}}$$

Best Case:

- All symbols in the file have the same frequency (uniform distribution).
- Huffman tree is **perfectly balanced**, giving minimal code lengths.
- Operations such as building the heap, constructing the tree, and generating codes are still bounded by **O(k log k)**, but small since k ≤ 256.
- Dominant factor: scanning and encoding the file.

  Time Complexity: O(n)

Worst Case:

- Highly skewed symbol frequencies (e.g., one symbol occurs almost all the time, others very few).
- Huffman tree becomes **unbalanced**, resulting in longer paths for rare symbols.
- Heap operations remain **O(k log k)**.
- Dominant factor remains encoding all bytes.

  Time Complexity: O(n + k log k) ≈ O(n)

(Since k is very small for byte data, best and worst cases are effectively the same for large files.**)**

Space Complexity:

The space requirements depend on the auxiliary structures used during compression and decompression:

Data Structures	Purpose	Space
Frequency Table	Stores frequency of each unique symbol	O(k)
Min-Heap	Temporary structure for building the tree	O(k)
Huffman Tree	Stores nodes for tree construction	O(k)
Code Table	Stores Huffman codes for each symbol	O(k)
Encoded Bitstring/ Output Buffer	Temporary storage of encoded data	O(n)

Overall Space Complexity:

$$S(n,k)= O(k)+O(k)+O(k)+O(k)+O(n)$$

The space used by structures dependent on k is small and constant, while the encoded bit stream grows with n. Therefore, the total space complexity is effectively linear in the input size.

Let S(n) represents the space complexity then, the space complexity is:

$$\boxed{S(n)=O(n)}$$

Best Case:

- Frequency table, Huffman tree, and code table occupy **O(k)** memory.
- Encoded bitstream is proportional to file size **O(n)**.

Space Complexity: $O(n + k) \approx O(n)$

Worst Case:

- The Huffman tree may be unbalanced, but it still has ≤ 2k nodes.
- Code table remains O(k).
- Encoded bitstream could slightly increase if many symbols have long codes.

Space Complexity: O(n + k) ≈ O(n)

Extent of Compression Achieved:

To evaluate the performance of Huffman Coding, multiple file types were tested under controlled conditions. Each file category included two variants:

1. Repetitive data (high redundancy)
2. Non-repetitive data (low redundancy)

This allowed us to analyze how file size, data structure, and redundancy patterns affect the compression ratio. Huffman coding relies on symbol frequency; therefore, the degree of compression is highly dependent on how often characters or byte sequences repeat within the file. Larger files and files containing structured or repetitive content typically provide more stable statistical patterns, enabling more efficient encoding.

File Type	Repetitive Data	Compression Ratio (%)
txt	Yes	67.47%
txt	No	37.30%
csv	Yes	44.42%
csv	No	25.13%
json	Yes	50.61%
json	No	25.54%
jpeg	Yes (solid color image)	4.37%
jpeg	No (noise-rich image)	0.07%

Observations:

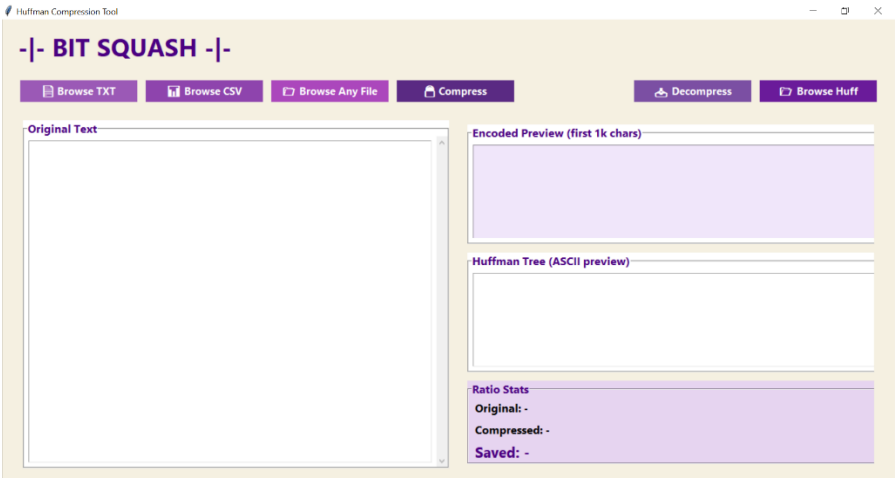
- **Compression Efficiency:** Files with high redundancy and larger sizes achieved the best compression ratios. Structured text files (TXT, CSV, JSON) benefited from recurring patterns, while small or non-repetitive files showed limited reduction.

- **Effect of File Size:** Larger files allow more accurate frequency estimation, enabling Huffman Coding to assign shorter codes effectively. Smaller files have fewer symbol repetitions, which reduces compression gains.

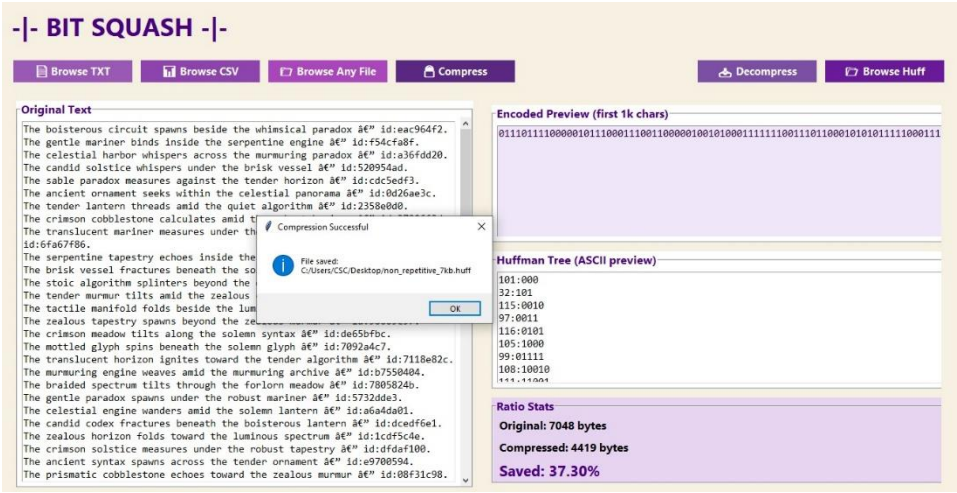
- **File Type Impact:** Text and structured data compress well due to predictable symbols and formatting. Image files (BMP/JPEG) show minor compression only if redundancy exists (e.g., solid-color or simple patterns), while complex or noise-rich images yield minimal gains.

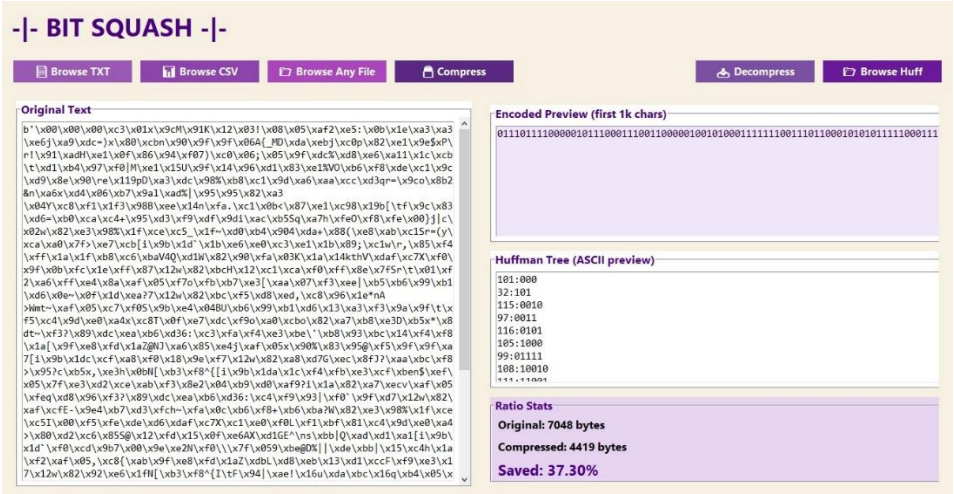
- **Practical Implications:** Huffman Coding is effective for diverse file types, but its efficiency depends on redundancy and size. It guarantees lossless reconstruction, making it ideal for text and structured datasets, while large or complex images may see only marginal compression.

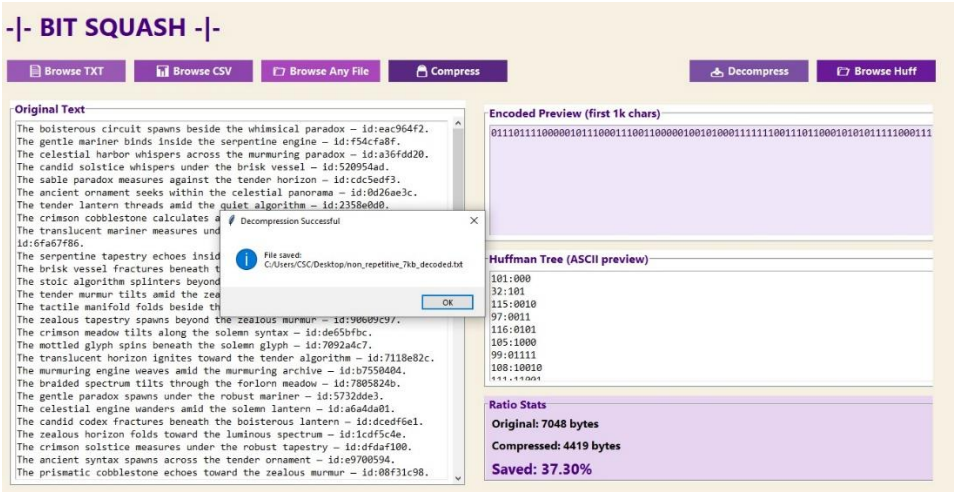
Sample Input and Output Files:



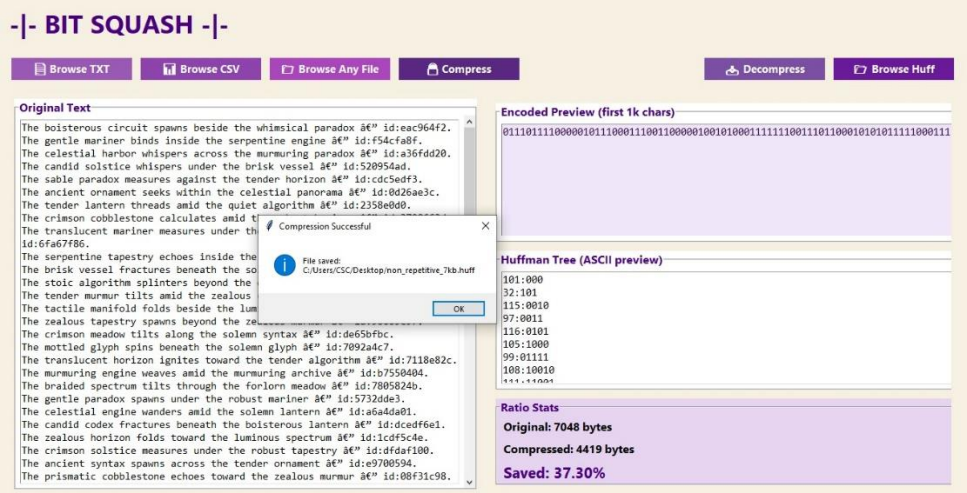
.txt File (for non-repetitive data):

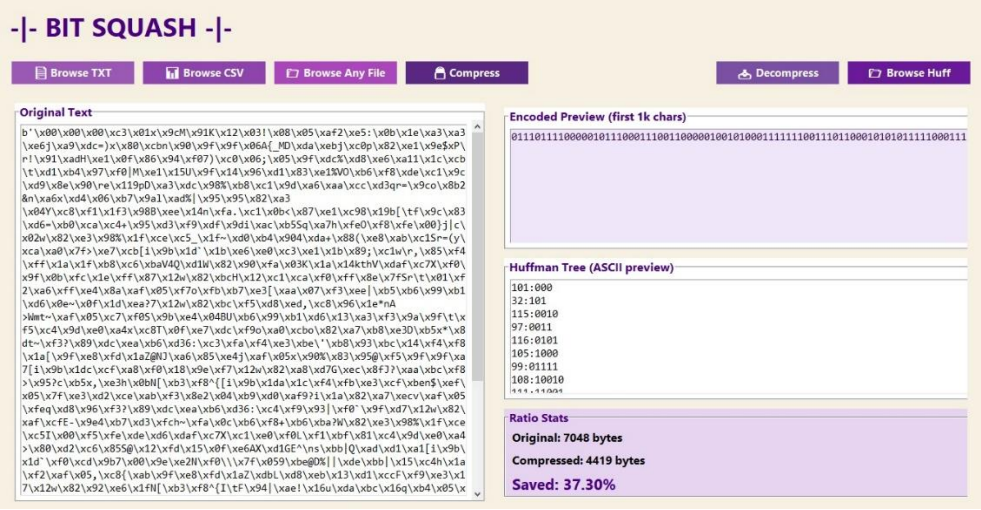


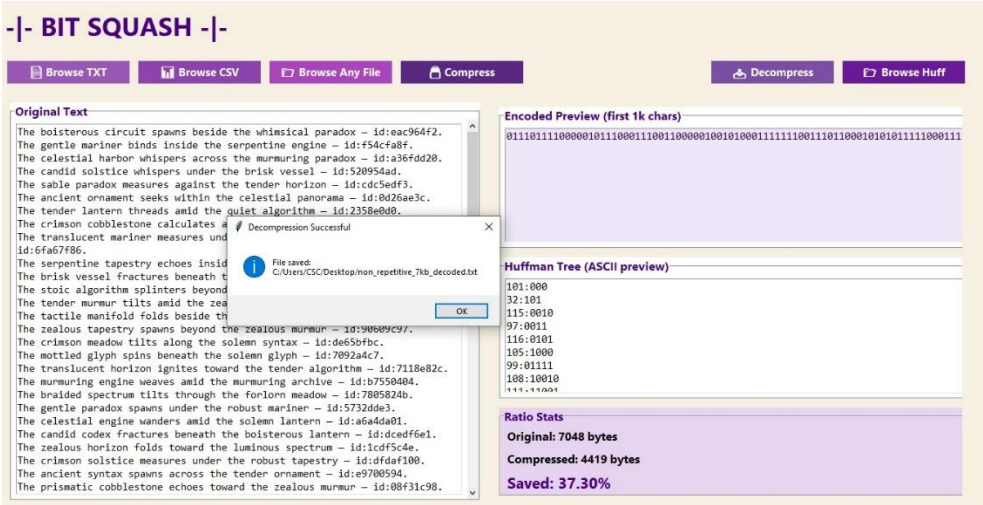




.csv File:


-|- BIT SQUASH -|-
Browse TXT
Browse CSV
Browse Any File
Compress
Decompress
Browse Huff
Original Text
Encoded Preview (first 1k chars)
Compression Successful
File saved:
C:/Users/CSC/Desktop/non_repetitive_7kb.huff
OK
Huffman Tree (ASCII preview)
101:000
32:101
115:0010
97:0011
116:0101
105:1000
99:01111
108:10010
Ratio Stats
Original: 7048 bytes
Compressed: 4419 bytes
Saved: 37.30%


-|- BIT SQUASH -|-
Browse TXT
Browse CSV
Browse Any File
Compress
Decompress
Browse Huff
Original Text
Encoded Preview (first 1k chars)
Huffman Tree (ASCII preview)
101:000
32:101
115:0010
97:0011
116:0101
105:1000
99:01111
108:10010
Ratio Stats
Original: 7048 bytes
Compressed: 4419 bytes
Saved: 37.30%



Conclusion:

Huffman Coding is a reliable lossless compression method that preserves all original data. It performs best on large, repetitive, or structured files and provides exact reconstruction. While compression is limited for small, highly random, or complex image files, the algorithm remains a practical and effective solution for multi-format file compression systems